

Fundación Universitaria CEIPA

Núcleo

Fundamentos de Software

Actividad 4-2: Serialización y persistencia (JSON)

Presentado por:

Guillermo León Ríos Franco

Carlos Arturo Molina Tomases

Docente:

Simón Peláez Loaiza

Ingeniería de Sistemas

2026

Fundamentos de la Serialización

1. ¿Qué es la Serialización? (La analogía del paquete)

Para entender la **serialización**, imaginemos que construimos un mueble de madera muy complejo y articulado en nuestra casa (este mueble representa un **Objeto en la memoria de Python**). Si necesitamos enviarle ese mueble a un amigo que vive en otra ciudad o guardarlo en una bodega pequeña (el disco duro o internet), no podemos mandarlo armado porque no cabe por las tuberías de la red ni por la puerta de la bodega.

- **La Serialización:** Es el proceso de desarmar el mueble con cuidado, organizar sus piezas en orden y meterlas en una caja de cartón plana con sus instrucciones de armado. Hemos convertido una estructura compleja en un **paquete plano y lineal** (una cadena de bytes o de texto).
- **La Deserialización:** Es el proceso inverso. Cuando el paquete llega a su destino, nuestro amigo abre la caja y, siguiendo las instrucciones, vuelve a armar el mueble idéntico a como estaba originalmente en nuestra memoria.

Es una **etapa obligatoria** porque los discos duros y las redes de internet solo entienden y transmiten flujos lineales de información (unos y ceros, o texto plano de corrido). No tienen la capacidad de almacenar directamente la "estructura viva" ni las conexiones en memoria de un objeto de Python.

2. ¿Por qué usamos JSON en lugar de Texto Plano Simple?

Aunque el texto plano simple (como un archivo separado por comas .csv o un .txt ordinario) sirve para guardar datos sueltos, se queda muy corto para estructuras complejas por las siguientes razones:

- **Soporte de Estructuras Jerárquicas:** El texto plano es bidimensional (filas y columnas). JSON permite anidar información de forma nativa (listas dentro de diccionarios, objetos dentro de otros objetos).
- **Tipado de Datos Estándar:** En texto plano simple, todo es texto. Si guardas un número 45000 o un booleano True, al leer el archivo plano todo vuelve como un string ("45000", "True"). JSON respeta y diferencia nativamente si un dato es un string, un número, un booleano (true/false) o un valor nulo (null).

- **Estándar Universal:** Está basado en texto legible por humanos, pero es interpretado de forma exacta por prácticamente cualquier lenguaje de programación en el mundo (Python, JavaScript, Java, C#, etc.), convirtiéndose en el idioma oficial para la comunicación con APIs y servidores web.
- En Python, un objeto creado a partir de una clase no puede convertirse directamente a JSON en todos los casos, debido a que el módulo json trabaja principalmente con estructuras de datos compatibles como diccionarios, listas, cadenas de texto, números y valores booleanos. Por esta razón, es necesario realizar una transformación intermedia del objeto a un diccionario. Este proceso permite representar los atributos del objeto mediante pares clave-valor y posteriormente serializar la información en formato JSON.
- El proceso inverso requiere leer la información almacenada en JSON, convertirla nuevamente en una estructura de Python y utilizar esos datos para reconstruir el objeto original. De esta manera, la deserialización no consiste únicamente en leer un archivo, sino en recuperar el estado de la información para que pueda volver a ser utilizada dentro del programa.

Paso 2: Simulación de Conversión Objeto a JSON y Viceversa

Tomando como base la clase provista por la guía, implementamos el proceso completo utilizando la librería nativa json de Python.

```

39
40
41 # =====
42 # B. DESERIALIZACIÓN (JSON a Objeto)
43 # =====
44 print("--- PROCESO DE DESERIALIZACIÓN ---")
45
46 # Imaginemos que acabamos de recibir la variable 'cadena_json' desde internet o el cliente
47 json_recibido = cadena_json
48
49 # 1. Convertir la cadena JSON de vuelta a un Diccionario de Python
50 # Usamos json.loads() para 'load string' (cargar desde cadena de texto)
51 diccionario_recuperado = json.loads(json_recibido)
52 print(f"1. JSON decodificado a Diccionario: {diccionario_recuperado}")
53 print(f" Tipo de dato: {type(diccionario_recuperado)}\n")
54
55 # 2. Usar la información del diccionario para instanciar un NUEVO objeto Transaccion
56 nuevo_objeto = Transaccion(
57     cliente_id=diccionario_recuperado["cliente_id"],
58     tipo=diccionario_recuperado["tipo"],
59     monto=diccionario_recuperado["monto"]
60 )
61 print(f"2. Nuevo Objeto POO instanciado exitosamente: {nuevo_objeto}")
62 print(f" Tipo de dato final: {type(nuevo_objeto)}")

```

```

Tipo de dato final: <class 'str'>

--- PROCESO DE DESERIALIZACIÓN ---
1. JSON decodificado a Diccionario: {'cliente_id': 'C001', 'tipo': 'CREDITO', 'monto': 50000.0}
Tipo de dato: <class 'dict'>

2. Nuevo Objeto POO instanciado exitosamente: Transacción [CREDITO] - ID: C001, Monto: 50000.0
Tipo de dato final: <class '__main__.Transaccion'>
PS C:\Users\k-rlo\Desktop\Python\Semana_4>

```

Análisis de la implementación

En la evidencia presentada se observa la aplicación práctica del proceso de serialización y deserialización. Inicialmente, los atributos del objeto se convierten en una estructura compatible con JSON y posteriormente se utiliza la librería json de Python para generar su representación serializada. Este proceso permite que la información deje de depender únicamente de la memoria temporal del programa.

Al realizar el proceso inverso, los datos pueden ser recuperados y utilizados nuevamente para reconstruir la información del objeto. Desde mi perspectiva como estudiante de Ingeniería de Sistemas, esta práctica permite entender cómo los objetos utilizados durante la ejecución de un software pueden transformarse en datos que posteriormente pueden ser almacenados, transportados o compartidos con otros sistemas.

Conclusiones sobre la Evidencia de Competencia

1. **Persistencia:** Gracias a este flujo (Objeto -> Diccionario -> JSON), el estado del software puede guardarse en archivos .json locales de forma persistente, de modo que al cerrar y abrir el programa los datos no se borren de la memoria RAM.
2. **Modularidad y Comunicación:** Este estándar permite que los objetos de negocio mapeados en Python puedan ser enviados a un servidor web front-end (hecho en lenguajes como JavaScript) sin ningún tipo de fricción o incompatibilidad de formatos.
3. **Interoperabilidad:** La serialización mediante JSON facilita el intercambio de información entre diferentes sistemas y tecnologías. Esto es importante en el desarrollo de software moderno, debido a que una aplicación puede estar compuesta por diferentes componentes desarrollados en distintos lenguajes de programación. El uso de un formato estándar permite que la información conserve una estructura comprensible y pueda ser procesada por diferentes aplicaciones.